LÖSUNGEN · LEHREREXEMPLAR

W5·1 – Der Wald: Binärbäume – Lösungen

Zum Arbeitsblatt `AQ_W5_AB1_Binaerbaeume.docx` · IDE: `AQ_W5_L5_Wald_LSG.txt` (Klassen Baum, Rekursiv, Akinator) · alle Bäume, Traversierungen, Rekursions- und Schlangentabellen, Rückgaben und Laufzeitfehler mit Java (javac 21) und exakten Nachbauten von BinTree, Queue und DynArray (Object-basiert, gleiche Exceptions) berechnet; jeder abgedruckte Java-Code wurde kompiliert und getestet · Schreibweise Klammerform: Wurzel(links,rechts), – = leerer Teilbaum

Teil 1 (DS 31)

Aufgabe 1 – Die alte Eiche (6 BE)

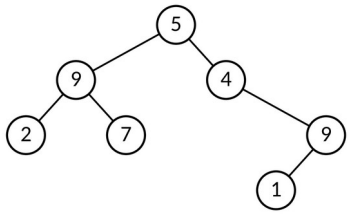
a) Wurzel **M**; Blätter **A, G, K, R, Z**; innere Knoten **M, F, B, H, T, W** (2 BE). Typischer Fehler: B als Blatt – B hat das linke Kind A.

b) Ebene von T: **1**, Ebene von K: **3**; Höhe von M: **4**; Höhe des Teilbaums mit der Wurzel F: **3** (2 BE).

c) Teilbaum mit der Wurzel H: **H, G, K**. Knoten je Ebene 0, 1, 2, 3: **1, 2, 4, 4** (zusammen 11 Knoten) (1 BE).

d) Ein Blatt ist nicht leer, beide Teilbäume sind leer und haben die Höhe 0. Nach der Definition ist seine Höhe 1 + max(0, 0) = 1 (1 BE).

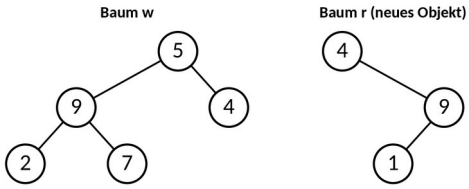
Aufgabe 2 – Vom Code zum Baum (7 BE)



a) Der Baum w ist der **zahlenbaum** aus Abbildung 1: `5(9(2,7),4(-,9(1,-)))` (2 BE). Zeile 5 klappt, weil `w.getLeft().getRight()` ein **leerer** Teilbaum ist: setItem macht ihn zum Blatt 7. Zeilen 7 und 8 verändern w, weil r eine **Referenz** auf den rechten Teilbaum von w ist.

Ausdruck	Wert	Begründung
`w.getRight().isLeaf()`	false	4 hat das rechte Kind 9
`w.getLeft().getRight().isLeaf()`	true	7 hat zwei leere Teilbäume
`(Integer) r.getRight().getItem() + 1`	10	Cast vor +: 9 + 1
`w.getRight().getLeft().isEmpty()`	true	linker Teilbaum von 4 ist leer

b) je Zeile ½ BE (2 BE).



c) w = `5(9(2,7),4)`, r = `4(-,9(1,-))`. Mit `new BinTree(4)` ist r ein **neues, eigenes Objekt** (ein Blatt 4), das mit w nichts zu tun hat. Die Zeilen 7 und 8 bauen deshalb nur r aus; der rechte Teilbaum von w bleibt das Blatt 4. In der Originalfassung zeigt r auf **denselben** Teilbaum wie `w.getRight()` – jede Änderung über r ist eine Änderung an w (2 BE).

d) Laufzeitfehler in **Zeile 7** (Java: IllegalStateException). Ohne Zeile 3 ist `w.getRight()` der **leere** Baum (nicht null!), Zeile 6 merkt ihn sich in r – das ist noch erlaubt. In Zeile 7 wird `r.getRight()` auf dem leeren Baum aufgerufen: getRight auf einem leeren Baum ist verboten (1 BE).

Aufgabe 3 – Vom Baum zum Code (5 BE)

a) z. B. (2 BE):

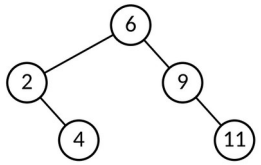
```
BinTree links = new BinTree(2);
links.setRight(new BinTree(4));
BinTree rechts = new BinTree(9);
rechts.setLeft(new BinTree(7));
rechts.setRight(new BinTree(11));
BinTree x = new BinTree(6);
x.setLeft(links);
x.setRight(rechts);
```

b) (2 BE):

```
BinTree x = new BinTree(6);
x.getLeft().setItem(2);
x.getLeft().getRight().setItem(4);
x.getRight().setItem(9);
x.getRight().getLeft().setItem(7);
x.getRight().getRight().setItem(11);
```

Beide Fassungen liefern `6(2(-,4),9(7,11))` (mit Java geprüft). In b) wird jeder **leere** Teilbaum durch setItem zum Blatt – die Reihenfolge ist wichtig: Erst muss 2 stehen, bevor `x.getLeft().getRight()` erreichbar ist. Andere Reihenfolgen in a) sind richtig, solange jeder Baum vor setLeft/setRight erzeugt ist.

c) `x.getRight().setLeft(new BinTree());` oder `x.getRight().getLeft().setEmpty();` → `6(2(-,4),9(-,11))` (1 BE). **Nicht** erlaubt: `setLeft(null)` (IllegalArgumentException).



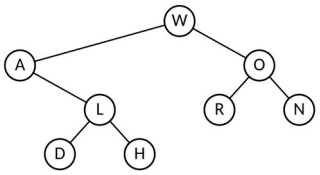
Aufgabe 4 – Rundgänge durch die Eiche (9 BE)

Traversierung	Folge der Knoten
Preorder	M F B A H G K T R W Z
Inorder	A B F G H K M R T W Z
Postorder	A B G K H F R Z W T M

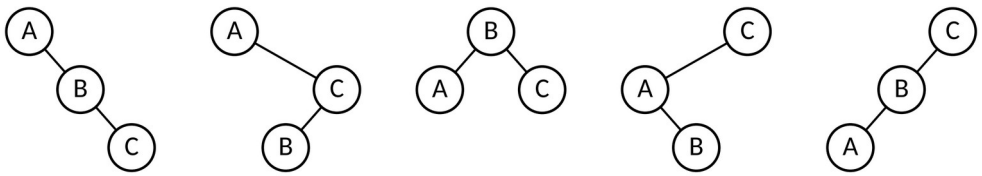
a) je Folge 1 BE (3 BE). **b)** Die Inorder ist **alphabetisch sortiert**. Vermutung: Für jeden Knoten stehen links nur kleinere, rechts nur größere Buchstaben – M ist ein **binärer Suchbaum** (DS 34) (1 BE).

c) Rekonstruktion (3 BE): Die Preorder liefert die Wurzel, die Inorder teilt in linken und rechten Teilbaum; dann rekursiv weiter.

Preorder	Inorder	Wurzel	links (in)	rechts (in)
WALDHORN	ADLHWRON	W	ADLH	RON
ALDH	ADLH	A	–	DLH
LDH	DLH	L	D	H
ORN	RON	O	R	N

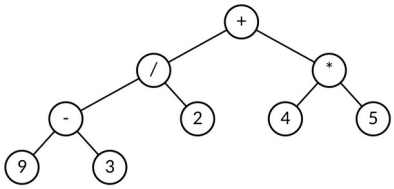


Postorder zur Kontrolle: **D H L A R N O W**. Die Rekonstruktion ist eindeutig (mit Java nachgebaut: wieder `W(A(-,L(D,H)),O(R,N))`).



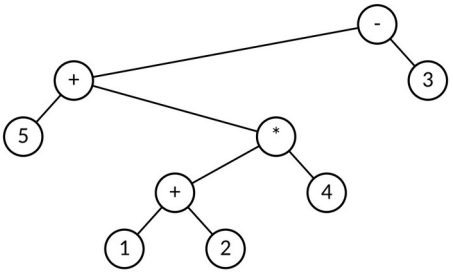
d) Alle **5** Bäume mit der Inorder A B C (zwei davon genügen für die Zeichnung) (2 BE). Die Inorder legt nur die Reihenfolge fest, nicht, welcher Knoten Wurzel ist – erst zusammen mit Pre- oder Postorder ist der Baum eindeutig.

Aufgabe 5 – Rechnende Bäume (9 BE)



a) Wurzel ist das **zuletzt** ausgewertete Rechenzeichen, das „+“ (Punkt vor Strich); links `(9 - 3) / 2`, rechts `4 * 5`. Wert: `(9 - 3) / 2 + 4 * 5` = 3 + 20 = **23** (2 BE). Kontrolle Postorder: `9 3 - 2 / 4 5 * +`.

b) Term: `6 * (1 + 2) - 8 / 4`, Wert **16** (18 − 2); Preorder `- * 6 + 1 2 / 8 4`, Postorder `6 1 2 + * 8 4 / -` (3 BE). Die Klammer um 1 + 2 ist nötig, sonst gilt Punkt vor Strich; 8 / 4 braucht keine Klammer.



c) Termbaum: `-(+(5,*(+(1,2),4)),3)` – Term `5 + (1 + 2) * 4 - 3`. Stapel (oben zuerst) nach jedem Zeichen; Wert **14** (3 BE):

Zeichen	Stapel
5	5
1	1, 5
2	2, 1, 5
+	3, 5
4	4, 3, 5
*	12, 5
+	17
3	3, 17
-	14

Bauen geht genauso: Zahl → neues Blatt auf den Stapel; Rechenzeichen → zwei Bäume vom Stapel holen (zuerst den **rechten**), als Teilbäume unter das Rechenzeichen hängen, den neuen Baum auflegen (mit Java geprüft: Postorder des Baums = `5 1 2 + 4 * + 3 -`).

d) Die Struktur des Baums legt fest, was zuerst gerechnet wird: Ein Rechenzeichen verknüpft immer die **fertigen Werte** seiner Teilbäume, tiefer liegende Knoten werden zuerst ausgewertet. Die Inorder verliert diese Struktur – z. B. ist `6 * 1 + 2 - 8 / 4` ohne Klammern nicht mehr der Term aus b). Pre- und Postorder sind dagegen ohne Klammern eindeutig (1 BE).

Aufgabe 6 – Level 5.1

Musterlösungen und häufige Fehler: `AQ_W5_L5_Wald_LSG.txt`, Klasse Baum – 11 Tests (basis 4, erweiterung 3, experte 4). Häufig: isLeaf vor isEmpty (Laufzeitfehler beim leeren Teilbaum), Höhe ohne + 1, Leerzeichen vergessen, Rechenzeichen mit == verglichen, spiegeln ohne Zwischenspeichern des linken Teilbaums.

Kontrollwerte aus dem Hauptprogramm (beispielBST `8(3(1,6(4,7)),10(-,14(13,-)))`): Knoten 9 · Preorder `8 3 1 6 4 7 10 14 13` · Inorder `1 3 4 6 7 8 10 13 14` · Postorder `1 4 7 6 3 13 14 10 8` · Höhe 4 · Blätter 4 · Summe 66 · Termbaum `3 + 4 * 5 - 2 = 21` · Postfix `3 4 + 5 2 - *` · gespiegelt `8(10(14(-,13),-),3(6(7,4),1))`.

Teil 2 (DS 32)

Aufgabe 7 – Wie hoch reicht der Ast? (6 BE)

a) Rekursionstabelle für `hoehe(t)` mit t = `3(1,6(4,7))` (4 BE); Ergebnis **3**:

Rekursionstiefe	Aufruf	l	r	Rückgabe
0	hoehe(3)	1	2	3

Rekursionstiefe	Aufruf	l	r	Rückgabe
1	hoehe(1)	0	0	1
2	hoehe(leer)	–	–	0
2	hoehe(leer)	–	–	0
1	hoehe(6)	1	1	2
2	hoehe(4)	0	0	1
3	hoehe(leer)	–	–	0
3	hoehe(leer)	–	–	0
2	hoehe(7)	0	0	1
3	hoehe(leer)	–	–	0
3	hoehe(leer)	–	–	0

Bewertung: Reihenfolge der Aufrufe und Tiefen 2 BE, Werte von l, r und Rückgaben 2 BE. Typische Fehler: leere Teilbäume weggelassen, Rückgabe des Blatts 0 statt 1, Erstaufruf mit Tiefe 1.

b) 11 Aufrufe, größte Rekursionstiefe **3**. Für den ganzen beispielBST (9 Knoten): **19** Aufrufe (größte Tiefe 4). Allgemein **2n + 1**: Jeder der n Knoten löst genau einen Aufruf aus, dazu kommt ein Aufruf für jeden leeren Teilbaum – jeder Knoten hat zwei Teilbaum-Plätze (2n), n − 1 davon sind von Knoten belegt (jeder Knoten außer der Wurzel hängt an genau einem Platz), also gibt es n + 1 leere Teilbäume: n + (n + 1) = 2n + 1 (mit Java für alle Testbäume geprüft) (2 BE).

Aufgabe 8 – Papier-Compiler im Wald (10 BE)

a) (3 BE) beispielBST 5, zahlenbaum 4, termbaum 3, leerer Baum / Blatt 0 / 0:

```
public static int anzahlInnere(BinTree b) {
    if (b.isEmpty()) {
        return 0;                                  // leerer Baum: kein Knoten
    }
    if (b.isLeaf()) {
        return 0;                                  // Blatt: kein innerer Knoten
    }
    return 1 + anzahlInnere(b.getLeft()) + anzahlInnere(b.getRight());
}
```

b) (3 BE) zahlenbaum: x = 9 → 2, x = 5 → 1, x = 3 → 0; leerer Baum → 0:

```
public static int zaehle(BinTree b, int x) {
    if (b.isEmpty()) {
        return 0;
    }
    int anzahl = zaehle(b.getLeft(), x) + zaehle(b.getRight(), x);
    int wert = (Integer) b.getItem();
    if (wert == x) {
        anzahl = anzahl + 1;
    }
    return anzahl;
}
```

Je 1 BE: Basisfall isEmpty zuerst · (a: Blatt-Fall) bzw. Cast (b) · rekursive Aufrufe für beide Teilbäume, Ergebnisse verwendet. In b) darf **nicht** wie im Suchbaum nur ein Teilbaum durchsucht werden. Gleichwertig: `return 1 + …` nur, wenn `!b.isLeaf()`.

c) Papier-Compiler (4 BE): Musterlösung aus `AQ_W5_L5_Wald_LSG.txt` – mit Java gegen eine unabhängige Prüfung für alle Testbäume und x = 0 … 15 getestet. zahlenbaum: enthaelt 7 → true, 8 → false; maximum zahlenbaum 9, beispielBST 14, leer -1.

```
public static boolean enthaelt(BinTree b, int x) {
    if (b.isEmpty()) {
        return false;
    }
    int wert = (Integer) b.getItem();
    if (wert == x) {
        return true;
    }
    return enthaelt(b.getLeft(), x) || enthaelt(b.getRight(), x);
}
```

```
public static int maximum(BinTree b) {
    if (b.isEmpty()) {
        return -1;                                  // kleiner als jeder Wert >= 0
    }
    int max = (Integer) b.getItem();
    int links = maximum(b.getLeft());
    int rechts = maximum(b.getRight());
    if (links > max) {
        max = links;
    }
    if (rechts > max) {
        max = rechts;
    }
    return max;
}
```

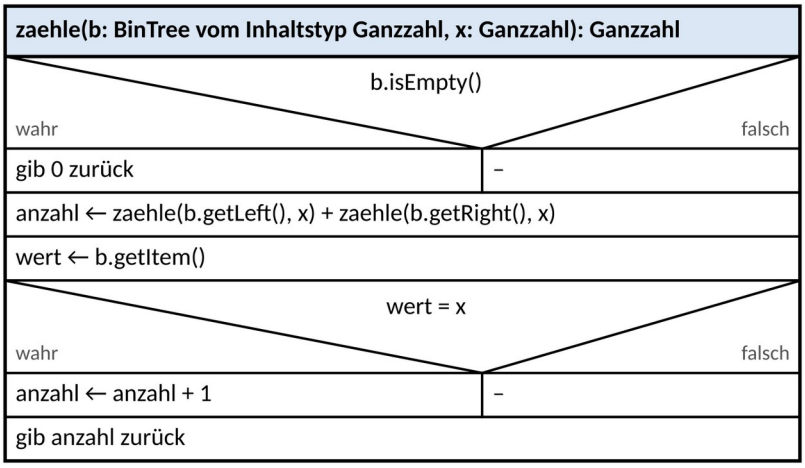
Häufige Papier-Fehler: Cast `(Integer)` vergessen (Compilerfehler), `return enthaelt(b.getLeft(), x);` ohne den rechten Teilbaum, maximum liefert für den leeren Baum 0 statt −1 (Test rot), fehlendes return am Ende.

Aufgabe 9 – Struktogramme im Unterholz (8 BE)

a) `geheim(zahlenbaum)` = **7**, `geheim(beispielBST)` = **13**. Beim zahlenbaum gibt es **9** Aufrufe: An Blättern endet die Rekursion sofort (kein Aufruf für ihre leeren Teilbäume), nur die leeren Teilbäume innerer Knoten (links von 4, rechts von 9) werden aufgerufen (3 BE).

b) geheim liefert den **größten Wert unter den Blättern** – nicht das Maximum aller Knoten: Werte innerer Knoten werden nie gelesen. Beim zahlenbaum liefert geheim 7, obwohl das Maximum 9 ist – beide 9 sind innere Knoten. Leerer Baum: **-1**. Weil alle Werte ≥ 0 sind, ist −1 kleiner als jeder Blattwert: Ein leerer Teilbaum „gewinnt“ den Vergleich l > r nie, das Ergebnis eines nicht leeren Baums ist immer ein echter Blattwert, und −1 zeigt eindeutig „kein Blatt“ an (2 BE).

c) Struktogramm zu zaehle (3 BE): Kopf mit Signatur 1 BE, Basisfall als Verzweigung 1 BE, rekursive Aufrufe als Zuweisungen und Zählen 1 BE. Gleichwertig: Wurzel zuerst prüfen, dann `gib anzahl + zaehle(…) + zaehle(…) zurück`.



Aufgabe 10 – Fehlerjagd im Unterholz (8 BE)

a) termbaum: **7** (richtig, jeder innere Knoten hat zwei Kinder). zahlenbaum: **Laufzeitfehler** (IllegalStateException) beim Aufruf für den leeren Teilbaum links von 4: isLeaf auf einem leeren Baum. Leerer Baum: sofort **Laufzeitfehler** (IllegalStateException). Der Basisfall „leer“ fehlt; korrigiert (3 BE) – liefert 7 bzw. 0:

```
public static int anzahlKnoten(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    if (b.isLeaf()) {
        return 1;
    }
    return 1 + anzahlKnoten(b.getLeft()) + anzahlKnoten(b.getRight());
}
```

Der Blatt-Fall darf bleiben, ist aber überflüssig (nur kürzer). Fehlerbild: Die Methode funktioniert genau dann, wenn jeder innere Knoten **zwei** Kinder hat.

b) Zeile 5: `getItem()` liefert Object, das nicht ohne Cast einer int-Variablen zugewiesen werden kann (javac: „incompatible types: Object cannot be converted to int“). Korrektur: `int w = (Integer) b.getItem();`. Dann: `summeGerade(zahlenbaum)` = **6** (2 + 4) (2 BE).

c) beispielBST: **3** statt 4; entarteter Baum: **5** (richtig). Fehler in **Zeile 8**: `return l;` muss `return l + 1;` heißen (bzw. `1 + l`) – ist der linke Teilbaum höher, wird der aktuelle Knoten nicht mitgezählt. Im entarteten Baum (rechte Kette) ist jeder linke Teilbaum leer: l = 0, r ≥ 0, die Bedingung l > r ist **nie** wahr – der fehlerhafte Zweig wird nie ausgeführt. Tests müssen beide Zweige einer Verzweigung erreichen (3 BE).

Aufgabe 11 – Level 5.2

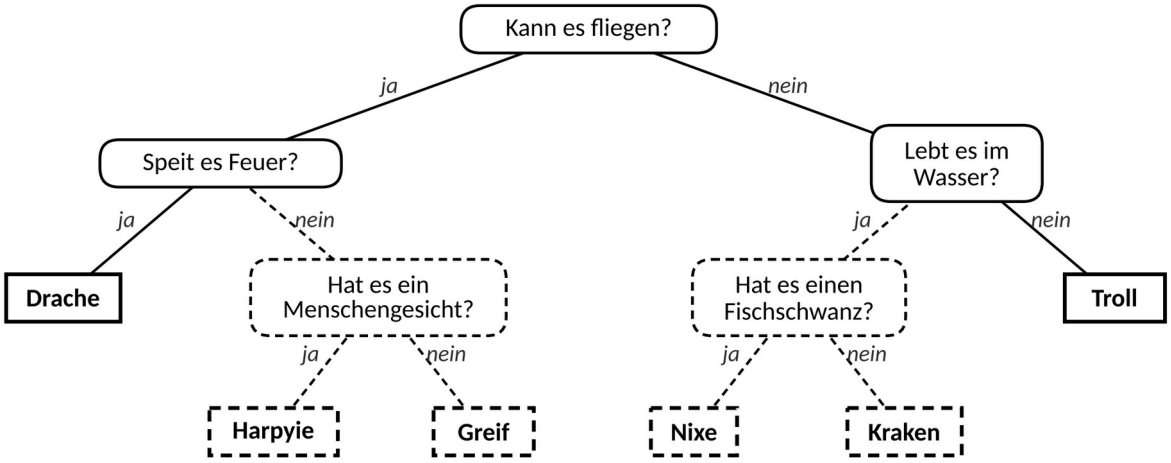
Musterlösungen: `AQ_W5_L5_Wald_LSG.txt`, Klasse Rekursiv – 8 Tests (basis 3, erweiterung 3, experte 2). Häufig: maximum mit 0 statt −1 begonnen, neue DynArray in jedem rekursiven Aufruf (Ergebnis bleibt leer), anzahlAufEbene ohne ebene − 1, pfadZu ohne Prüfung auf "-" (liefert "L-").

Kontrollwerte aus dem Hauptprogramm (zahlenbaum `5(9(2,7),4(-,9(1,-)))`): enthaelt 7: true · Maximum 9 · größer als 4: 4 · sammleInorder [2, 9, 7, 5, 4, 1, 9] · blaetter [2, 7, 1] · Knoten auf den Ebenen 0 bis 4: 1 2 3 1 0 · pfadZu 1: "RRL".

Teil 3 – Kür (DS 33)

Aufgabe 12 – Der Akinator rät und lernt (9 BE)

a) raten("jn") = **Greif** · raten("nn") = **Troll** · raten("n") = **?** · raten("nnnj") = **Troll** (2 BE). "n" endet an der Frage „Lebt es im Wasser?“ – zu wenige Antworten; bei "nnnj" werden die überzähligen Antworten ignoriert.



Baum nach beiden lernen-Aufrufen (neue Knoten gestrichelt)

b) (3 BE) Klammerform: `Kann es fliegen?(Speit es Feuer?(Drache,Hat es ein Menschengesicht?(Harpyie,Greif)),Lebt es im Wasser?(Hat es einen Fischschwanz?(Nixe,Kraken),Troll))`. Je Aufruf: Blatt wird zur Frage 1 BE, neues Monster links (ja) und altes rechts 1 BE; Gesamtbild 1 BE.

c) raten("jnj") = **Harpyie** · raten("njn") = **Kraken** · raten("nj") = **?** ("nj" führt jetzt zu einer Frage) · anzahlMonster() = **6**, anzahlFragen() = **5** (2 BE).

d) Am Anfang: 1 Monster, 0 Fragen – die Aussage gilt. Jeder lernen-Aufruf macht aus **einem** Blatt (Monster) **eine** Frage und hängt **zwei** Blätter an: Fragen + 1 und Monster − 1 + 2 = Monster + 1. Die Differenz Monster − Fragen bleibt 1 (Induktion) (2 BE). Kontrolle: 5/4 nach dem ersten, 6/5 nach dem zweiten Aufruf. Allgemein: Jeder innere Knoten hat genau zwei Kinder („voller“ Binärbaum).

Aufgabe 13 – Ebene für Ebene – mit der Schlange (7 BE)

a) levelorder(zahlenbaum) = `"5 9 4 2 7 9 1 "` (4 BE):

Durchlauf	entnommen	text	Schlange (vorne → hinten)
Start	–	""	5
1	5	"5 "	9, 4
2	9	"5 9 "	4, 2, 7
3	4	"5 9 4 "	2, 7, 9
4	2	"5 9 4 2 "	7, 9
5	7	"5 9 4 2 7 "	9
6	9	"5 9 4 2 7 9 "	1
7	1	"5 9 4 2 7 9 1 "	–

b) Beim zahlenbaum stehen höchstens **3** Teilbäume in der Schlange (nach Durchlauf 2: 4, 2, 7). Beim vollständigen Baum mit 4 Ebenen höchstens **8**: Wird der letzte Knoten von Ebene 2 entnommen, stehen genau alle 8 Knoten der untersten Ebene in der Schlange – allgemein so viele wie die breiteste Ebene (mit Java geprüft) (2 BE).

c) Rekursion arbeitet einen Teilbaum **vollständig** ab, bevor sie zum Nachbarn wechselt (Tiefe zuerst) – die Reihenfolge der Aufrufe verwaltet der Aufrufstapel automatisch. Bei der Levelorder müssen Knoten

verschiedener Teilbäume abwechselnd besucht werden: Die Kinder werden hinten angestellt und erst ausgegeben, wenn die ganze aktuelle Ebene fertig ist – genau das FIFO-Prinzip der Schlange (1 BE).

Aufgabe 14 – Level 5.3

Musterlösung: `AQ_W5_L5_Wald_LSG.txt`, Klasse Akinator mit privaten rekursiven Hilfsmethoden – 8 Tests (basis 3, erweiterung 3, experte 2). Häufig: `antworten.charAt(0)` ohne vorherige Längenprüfung (Laufzeitfehler bei ""), Zeichen mit "j" statt 'j' verglichen, lernen ersetzt das Blatt durch einen neuen Baum, statt es mit setItem/setLeft/setRight umzubauen (die Änderung kommt im Attribut nicht an), levelorder reiht leere Teilbäume ein (getItem auf leerem Baum).

Kontrollwerte aus dem Hauptprogramm: Monster 4, Fragen 3 · "nj" → Kraken · "nn" → Troll · nach lernen("nn", "Golem", "Ist es aus Stein?", true): "nnj" → Golem, Monster 5, Fragen 4 · Ebenenweise `Kann es fliegen? Speit es Feuer? Lebt es im Wasser? Drache Greif Kraken Ist es aus Stein? Golem Troll`.

Für Schnelle

- 16 Monster: **mindestens 4** Fragen bis zum schwierigsten Monster – bei einem ausgeglichenen Baum ($2^4 = 16$ Blätter auf einer Ebene, jede Frage halbiert die Kandidaten); **höchstens 15** Fragen bei einer Kette (jede Frage schließt nur ein Monster aus). Beide Werte mit Java durch Aufbau mit lernen geprüft.
- spiegeln vertauscht in jedem Knoten links und rechts: Die Fragen bleiben, aber „ja“ führt jetzt zu den Monstern des Nein-Zweigs. raten liefert nur noch richtige Monster, wenn man jede Frage verneint statt bejaht (bzw. die Konvention auf „ja = rechts“ ändert).
- Team-Wettbewerb: Kontrolle mit anzahlMonster() = anzahlFragen() + 1 und hoehe(getBaum()) (= längster Frageweg + 1).